

Module 7.2: Built-in Pipes

Learning Objectives:

- Master text transformation pipes (uppercase, lowercase, titlecase, slice)
- Learn number formatting with date, number, percent, and currency pipes
- Understand pipe parameters and customization options
- Apply pipes in practical, real-world scenarios

1. Text Transformation Pipes

1.1 Uppercase Pipe

The `uppercase` pipe transforms text to all uppercase letters.

Component TypeScript:

```
export class TextTransformComponent {  
  title: string = 'angular pipes tutorial';  
  message: string = 'Welcome to Angular Development!';  
}
```

Component Template:

```
<p>Original: {{ title }}</p>  
<p>Uppercase: {{ title | uppercase }}</p>  
<p>Message: {{ message | uppercase }}</p>
```

Output:

Original: angular pipes tutorial

Uppercase: ANGULAR PIPES TUTORIAL

Message: WELCOME TO ANGULAR DEVELOPMENT!

1.2 Lowercase Pipe

The `lowercase` pipe transforms text to all lowercase letters.

Component TypeScript:

```
export class TextTransformComponent {  
  heading: string = 'IMPORTANT ANNOUNCEMENT';  
  email: string = 'User@Example.COM';  
}
```

Component Template:

```
<p>Original: {{ heading }}</p>  
<p>Lowercase: {{ heading | lowercase }}</p>  
<p>Email: {{ email | lowercase }}</p>
```

Output:

Original: IMPORTANT ANNOUNCEMENT

Lowercase: important announcement

Email: user@example.com

1.3 Titlecase Pipe

The `titlecase` pipe capitalizes the first letter of each word.

Component TypeScript:

```
export class TextTransformComponent {  
  bookTitle: string = 'angular development for beginners';  
  courseName: string = 'INTRODUCTION TO WEB DEVELOPMENT';  
}
```

Component Template:

```
<p>Book: {{ bookTitle | titlecase }}</p>  
<p>Course: {{ courseName | titlecase }}</p>
```

Output:

Book: Angular Development For Beginners
Course: Introduction To Web Development

1.4 Slice Pipe

The `slice` pipe extracts a section of a string or array. It works similar to JavaScript's `slice()` method.

Syntax:

```
{{ value | slice:start:end }}
```

Component TypeScript:

```
export class SliceComponent {  
  text: string = 'The quick brown fox jumps over the lazy dog';  
  colors: string[] = ['red', 'blue', 'green', 'yellow', 'orange', 'purple'];  
}
```

Component Template:

```
<!-- String slicing -->  
<p>First 10 chars: {{ text | slice:0:10 }}</p>  
<p>From index 4 to 19: {{ text | slice:4:19 }}</p>
```

```
<p>Last 8 chars: {{ text | slice:-8 }}</p>

<!-- Array slicing -->
<p>First 3 colors: {{ colors | slice:0:3 }}</p>
<p>Last 2 colors: {{ colors | slice:-2 }}</p>
```

Output:

First 10 chars: The quick
From index 4 to 19: quick brown fox
Last 8 chars: lazy dog

First 3 colors: red,blue,green
Last 2 colors: orange,purple

2. Date Pipe

The `date` pipe formats date values according to locale rules or custom format strings.

2.1 Common Date Formats

Component TypeScript:

```
export class DateComponent {
  today: Date = new Date();
  birthday: Date = new Date(1995, 5, 15); // June 15, 1995
}
```

Component Template:

```
<p>Default: {{ today | date }}</p>
<p>Short: {{ today | date:'short' }}</p>
<p>Medium: {{ today | date:'medium' }}</p>
<p>Long: {{ today | date:'long' }}</p>
```

```
<p>Full: {{ today | date:'full' }}</p>
<p>Short Date: {{ today | date:'shortDate' }}</p>
<p>Long Date: {{ today | date:'longDate' }}</p>
```

Output:

Default: Jan 29, 2026

Short: 1/29/26, 10:30 AM

Medium: Jan 29, 2026, 10:30:45 AM

Long: January 29, 2026 at 10:30:45 AM GMT-5

Full: Thursday, January 29, 2026 at 10:30:45 AM Eastern Standard Time

Short Date: 1/29/26

Long Date: January 29, 2026

2.2 Custom Date Formats

You can create custom date formats using format strings:

Symbol	Meaning	Example
yyyy	4-digit year	2026
yy	2-digit year	26
MMMM	Full month name	January
MMM	Short month name	Jan
MM	2-digit month	01
dd	2-digit day	29
EEEE	Full day name	Thursday
EEE	Short day name	Thu
HH	24-hour (00-23)	14
h	12-hour (1-12)	2

Symbol	Meaning	Example
mm	Minutes	30
ss	Seconds	45
a	AM/PM	PM

Component Template:

```

<p>Custom 1: {{ today | date:'dd/MM/yyyy' }}</p>
<p>Custom 2: {{ today | date:'MMMM d, yyyy' }}</p>
<p>Custom 3: {{ today | date:'EEEE, MMM d' }}</p>
<p>Custom 4: {{ today | date:'h:mm a' }}</p>
<p>Custom 5: {{ today | date:'yyyy-MM-dd HH:mm:ss' }}</p>

```

Output:

Custom 1: 29/01/2026
 Custom 2: January 29, 2026
 Custom 3: Thursday, Jan 29
 Custom 4: 10:30 AM
 Custom 5: 2026-01-29 10:30:45

3. Number Formatting Pipes

3.1 Number Pipe

The `number` pipe formats numbers with decimal points and thousands separators.

Syntax:

```
{{ value | number:'minIntegerDigits.minFractionDigits-maxFractionDigits' }}
```

Component TypeScript:

```
export class NumberComponent {  
  price: number = 1234.567;  
  quantity: number = 50;  
  rating: number = 4.8;  
}
```

Component Template:

```
<p>Default: {{ price | number }}</p>  
<p>1 decimal: {{ price | number:'1.1-1' }}</p>  
<p>2 decimals: {{ price | number:'1.2-2' }}</p>  
<p>Min 3 digits: {{ quantity | number:'3.0-0' }}</p>  
<p>Rating: {{ rating | number:'1.1-1' }}</p>
```

Output:

Default: 1,234.567
1 decimal: 1,234.6
2 decimals: 1,234.57
Min 3 digits: 050
Rating: 4.8

3.2 Currency Pipe

The `currency` pipe formats numbers as currency values with currency symbol and locale-specific formatting.

Syntax:

```
{{ value | currency:'currencyCode':'display':'digitsInfo' }}
```

Component TypeScript:

```
export class CurrencyComponent {  
  productPrice: number = 49.99;  
  totalAmount: number = 1234.50;  
  discount: number = 15;  
}
```

Component Template:

```
<p>Default: {{ productPrice | currency }}</p>  
<p>USD Symbol: {{ productPrice | currency:'USD':'symbol' }}</p>  
<p>USD Code: {{ productPrice | currency:'USD':'code' }}</p>  
<p>EUR: {{ productPrice | currency:'EUR':'symbol' }}</p>  
<p>GBP: {{ productPrice | currency:'GBP':'symbol' }}</p>  
<p>Total: {{ totalAmount | currency:'USD':'symbol':'1.2-2' }}</p>
```

Output:

Default: \$49.99
USD Symbol: \$49.99
USD Code: USD49.99
EUR: €49.99
GBP: £49.99
Total: \$1,234.50

3.3 Percent Pipe

The `percent` pipe formats numbers as percentages.

Important: The pipe multiplies the value by 100 and adds the % symbol.

Component TypeScript:


```
export class PercentComponent {  
  discount: number = 0.25;    // 25%  
  completion: number = 0.875; // 87.5%  
  growth: number = 0.12;     // 12%  
}
```

Component Template:

```
<p>Discount: {{ discount | percent }}</p>  
<p>Completion: {{ completion | percent:'1.1-1' }}</p>  
<p>Growth: {{ growth | percent:'1.0-0' }}</p>
```

Output:

Discount: 25%
Completion: 87.5%
Growth: 12%

4. Pipe Parameters and Chaining

4.1 Using Pipe Parameters

Many pipes accept parameters to customize their behavior. Parameters are passed after a colon (:):

Component TypeScript:

```
export class ParametersComponent {  
  price: number = 1234.567;  
  today: Date = new Date();  
  longText: string = 'Angular provides a comprehensive solution for building web applications';  
}
```

Component Template:

```
<!-- Currency pipe with parameters -->
<p>US Format: {{ price | currency:'USD':'symbol':'1.2-2' }}</p>

<!-- Date pipe with different formats -->
<p>Short: {{ today | date:'short' }}</p>
<p>Custom: {{ today | date:'dd/MM/yyyy HH:mm' }}</p>

<!-- Slice pipe with start and end positions -->
<p>Excerpt: {{ longText | slice:0:40 }}...</p>
```

Output:

US Format: \$1,234.57

Short: 1/29/26, 10:15 AM

Custom: 29/01/2026 10:15

Excerpt: Angular provides a comprehensive solu...

4.2 Chaining Multiple Pipes

You can apply multiple pipes in sequence. Each pipe processes the output of the previous pipe:

Component TypeScript:

```
export class ChainingComponent {
  birthDate: Date = new Date(1995, 5, 15);
  description: string = 'welcome to angular pipes tutorial';
}
```

Component Template:

```
<p>Birth Date: {{ birthDate | date:'fullDate' | uppercase }}</p>
<p>Description: {{ description | titlecase | slice:0:20 }}</p>
```

Output:

Birth Date: THURSDAY, JUNE 15, 1995

Description: Welcome To Angular P

The execution order is left to right: `birthDate` → `date` pipe → `uppercase` pipe

5. Practical Example: Product Card

Component TypeScript:

```
export class ProductComponent {
  product = {
    name: 'wireless bluetooth headphones',
    price: 149.99,
    originalPrice: 199.99,
    rating: 4.7,
    reviews: 1543,
    inStock: true,
    releaseDate: new Date(2025, 10, 20),
    description: 'High-quality wireless headphones with active noise cancellation and 30-hour battery life. Perfect for music lovers and professionals.'
  };

  getDiscount(): number {
    return (this.product.originalPrice - this.product.price) / this.product.originalPrice;
  }
}
```

Component Template:

```
<div class="product-card">
  <h2>{{ product.name | titlecase }}</h2>

  <div class="price-section">
    <span class="current-price">{{ product.price | currency:'USD':'symbol':'1.2-2' }}</span>
    <span class="original-price">{{ product.originalPrice | currency:'USD':'symbol':'1.2-2' }}
  </span>
```

```

    <span class="discount">Save {{ getDiscount() | percent:'1.0-0' }}</span>
  </div>

  <div class="rating">
    <span>Rating: {{ product.rating | number:'1.1-1' }} / 5</span>
    <span>({{ product.reviews | number:'1.0-0' }} reviews)</span>
  </div>

  <p class="description">{{ product.description | slice:0:100 }}...</p>

  <p class="release-date">
    Released: {{ product.releaseDate | date:'MMMM d, yyyy' }}
  </p>

  <p class="stock-status">
    Status: {{ product.inStock ? 'In Stock' : 'Out of Stock' | uppercase }}
  </p>
</div>

```

Output:

Wireless Bluetooth Headphones

\$149.99 ~~\$199.99~~ **Save 25%**

Rating: 4.7 / 5 (1,543 reviews)

High-quality wireless headphones with active noise cancellation and 30-hour battery life. Perfect...

Released: November 20, 2025

Status: IN STOCK

6. Best Practices

Practice	Recommendation	Reason
Prefer pure pipes	Use pure pipes by default	

Practice	Recommendation	Reason
		Better performance and predictability
Limit chaining	Chain 2-3 pipes maximum	Maintains readability
Use specific formats	Specify date/number formats explicitly	Consistent display across locales
Avoid complex logic	Keep transformations simple	Move complex logic to components
Document custom usage	Add comments for complex pipe chains	Improves maintainability

7. Summary

In this module, we explored Angular's built-in pipes for common transformations:

- **Text Pipes:** uppercase, lowercase, titlecase, slice for text manipulation
- **Date Pipe:** Format dates with predefined or custom formats
- **Number Pipes:** currency, number, and percent for numeric formatting
- **Pipe Parameters:** Customize pipe behavior with parameters
- **Pipe Chaining:** Combine multiple pipes for complex transformations
- **Best Practices:** Use pure pipes, limit chaining, and keep logic simple

Next Steps: In the next module, we'll learn how to create custom pipes to handle specific transformation requirements that built-in pipes don't cover.

8. Practice Exercises

Exercise 1: Employee Directory

Create an employee directory component that displays:

- Employee name in titlecase
- Email in lowercase
- Salary in USD currency format
- Hire date in 'MMMM d, yyyy' format
- Performance rating as percentage

Exercise 2: Blog Post Preview

Create a blog post component with:

- Title in titlecase
- Author name in uppercase
- Publish date in 'MMM d, yyyy' format
- Content preview using slice pipe (first 150 characters)
- Read time in minutes

Exercise 3: Shopping Cart

Build a shopping cart that shows:

- Product names in titlecase
- Individual prices in currency format
- Quantity with number pipe
- Discount percentage
- Total price with currency pipe
- Order date formatted as 'EEEE, MMMM d, yyyy'